

Práctica 4 – Reconocimiento de Escenas con redes convolucionales neuronales

García Santa, Carlos – González Gallego Miguel Ángel

1 PREGUNTAS TAREAS

1.1 Estudie la variación en rendimiento para diferentes valores del batch_size = [8, 16, 32, 64] y 50 épocas. Razone sobre los resultados observados.

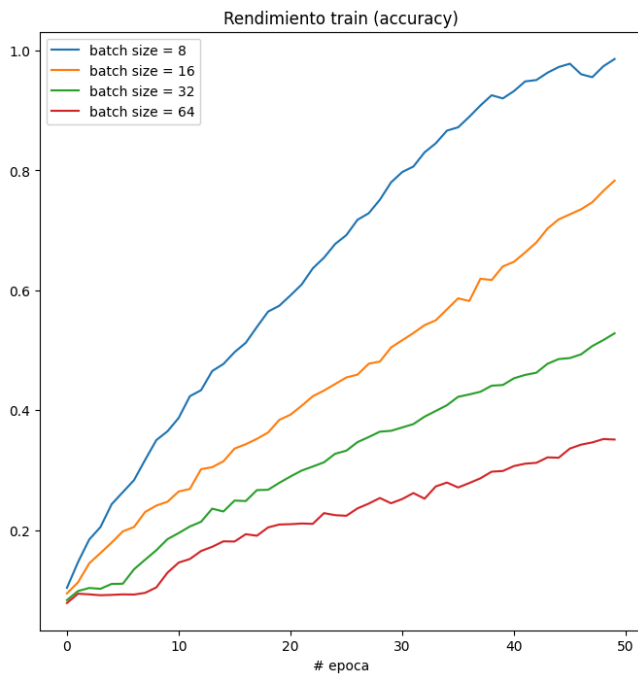


Fig. 1. Precisión en train para todos los tamaños de batch



Fig. 2. Precisión en test para todos los tamaños de batch

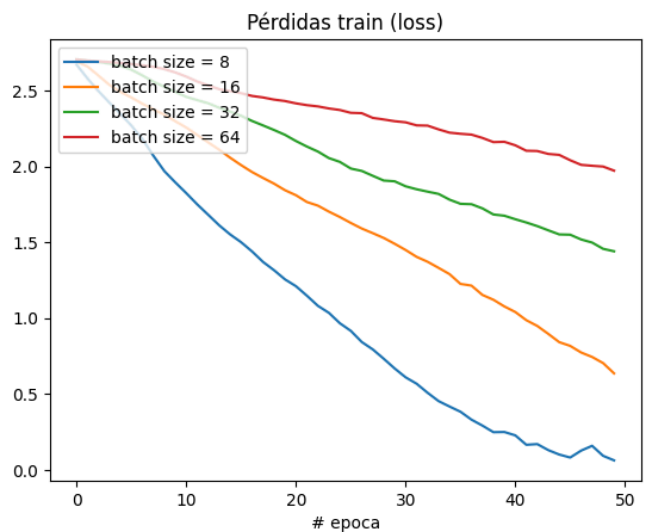


Fig. 3. Pérdidas en train para todos los tamaños de batch

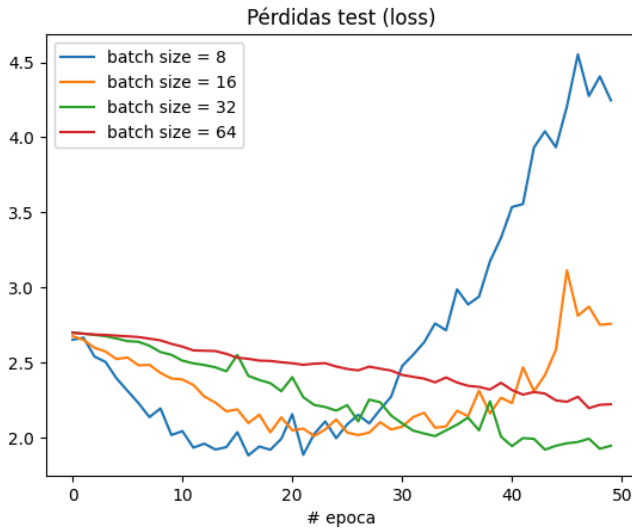


Fig. 4. Pérdidas en test para todos los tamaños de batch

Analizando el rendimiento y la pérdida para todos los tamaños de batch en los conjuntos de entrenamiento y prueba podemos observar dos tendencias claras. Por un lado, en las Fig. 1 y Fig. 2 el rendimiento o precisión de clasificación tanto para el conjunto de datos de entrenamiento como para el de validación se obtiene una precisión de clasificación mayor cuanto menor sea el tamaño de batch, siendo esto especialmente notable en el conjunto de entrenamiento donde para un batch de tamaño 8 se obtiene un rendimiento de clasificación cercano a 1, mientras que para un tamaño de 64 el rendimiento tan solo alcanza 0.35. En el conjunto de validación esto no es tan pronunciado llegando a aproximadamente un 0.38 de precisión el tamaño 8 y 0.26 el tamaño 64.

La otra tendencia corresponde con la pérdida, en este caso analizando la Fig. 3 tenemos que en el conjunto de entrenamiento la pérdida es menor cuanto menor sea el tamaño de batch, teniendo una pérdida cercana a 0 para tamaño 8 y alrededor de 2.1 para el tamaño 64, situándose entre medias las pérdidas para los tamaños 16 y 32, siendo aproximadamente 1.6 y 0.75 respectivamente. De forma opuesta, en la Fig. 4 en las pérdidas para el conjunto de validación cuanto menor sea el tamaño de batch mayor es la pérdida final, sin embargo, hay que destacar que esto sucede al final de la ejecución con 50 épocas ya que la tendencia inicial hasta 25 épocas para todos los tamaños de batch es decrecer, pero para los tamaños 8 y 16 a partir de estas 25 épocas la pérdida crece y acaba siendo mayor que para los tamaños 32 y 64. Esta tendencia nos indica que la red neuronal está sobreajustándose para los tamaños de batch 8 y 16 cuando el número de épocas es mayor a 25, esto sucede porque con un número pequeño de batch se realiza un mayor número de descensos por gradiente en cada época con lo cual el ajuste al conjunto de entrenamiento es mayor como se ve en el rendimiento en la Fig. 1 y en las pérdidas en la Fig. 3, esto desemboca en una peor generalización a partir de un determinado número de épocas como se ha observado en la Fig. 4.

Dado este sobreajuste observado en los tamaños de batch

8 y 16 para el número de épocas solicitado, se considera elegir tamaños de batch cuya pérdida continúe disminuyendo, con lo cual la elección de batch será 32 o 64 ya que como se ha mencionado estos son los tamaños de batch cuya pérdida en la validación decrece de forma monótona. Por lo tanto, atendiendo tanto al rendimiento como a la pérdida en la validación, es decir, analizando la capacidad de generalización de la red neuronal vemos que el mejor batch será el de tamaño 32 que tiene una pérdida menor y un rendimiento mayor que el tamaño 64.

1.2 Estudie la variación en rendimiento para diferentes tamaños de la imagen de entrada ([32x32, 64x64, 128x128, 224x224]) durante 50 épocas. Elija un valor de batch_size acorde a sus conclusiones en la pregunta anterior 4.1. Razone porque obtiene los resultados que observa.

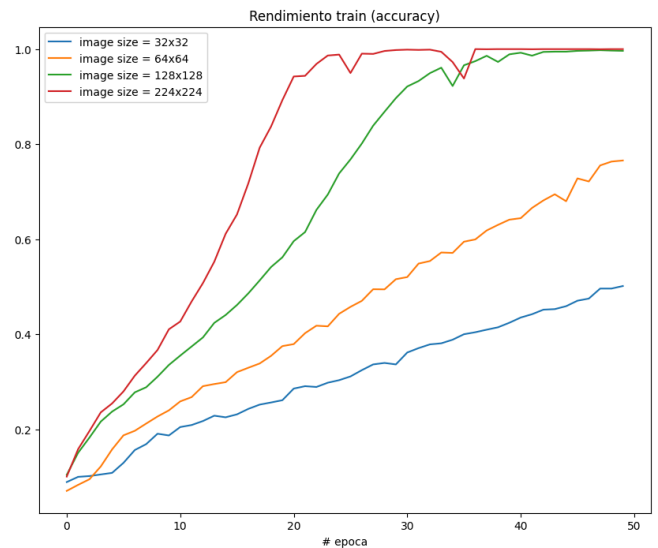


Fig. 5. Precisión en train para todos los tamaños de imagen



Fig. 6. Precisión en test para todos los tamaños de imagen

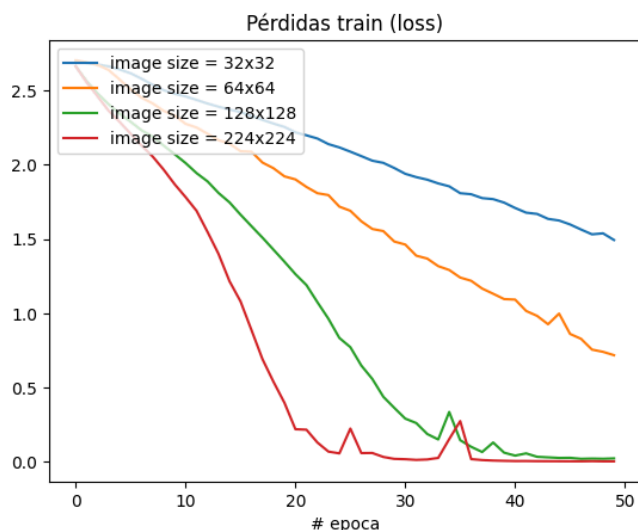


Fig. 7. Pérdidas en train para todos los tamaños de imagen

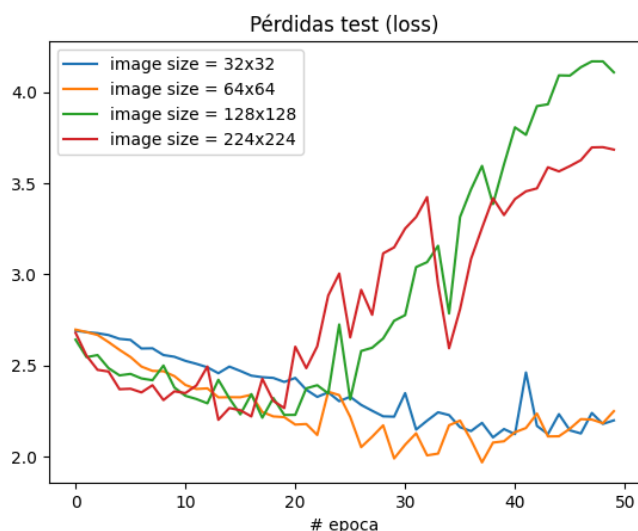


Fig. 8. Pérdidas en test para todos los tamaños de imagen

Empleando el tamaño de batch 32, analizamos los resultados obtenidos para diferentes tamaños de imagen, concretamente 32x32, 64x64, 128x128 y 224x224, podemos observar tendencias similares a las discutidas anteriormente respecto a los tamaños de batch. A medida que aumenta el tamaño de la imagen, la cantidad de información y detalles presentes en los datos también aumenta, lo que tiene un impacto significativo en el rendimiento y la pérdida tanto en el conjunto de entrenamiento como en el de validación.

En relación al rendimiento, observamos en la Fig. 5 a medida que crece el tamaño de la imagen la precisión en el conjunto de entrenamiento aumenta considerablemente. Por ejemplo, para imágenes de 32x32, la red alcanza valores de rendimiento estables en etapas tempranas del entrenamiento, mientras que en imágenes de 224x224 y 128x128 el rendimiento llega a alcanzar 1, esto nos indica que con imágenes de mayor tamaño, la red es capaz de ajustar los datos de entrenamiento con una precisión perfecta. Sin embargo, al analizar el comportamiento en las pérdidas de la validación en la

Fig. 8, se evidencia que esta capacidad de ajuste no se traduce en una mejora generalizada. En particular, vemos como para imágenes de mayor tamaño de 128x128 y 224x224, la pérdida en validación crece significativamente a medida que avanza el entrenamiento, especialmente después de la mitad de las épocas, indicando un sobreajuste claro.

Este fenómeno puede explicarse por la naturaleza de las imágenes de mayor tamaño, que aunque aportan más detalles y variedad de características, también introducen ruido y patrones específicos del conjunto de entrenamiento que no son relevantes o generalizables al conjunto de validación. La red, al disponer de más información, tiende a aprender estos detalles no relevantes, ajustándose excesivamente a los datos de entrenamiento y perdiendo capacidad de generalización.

Comparando estos resultados con los obtenidos para imágenes más pequeñas, como 32x32 y 64x64, encontramos que estas últimas permiten un equilibrio más estable entre rendimiento y pérdida. La pérdida en validación decrece de forma progresiva y alcanza valores mínimos al final del entrenamiento, sin mostrar el incremento que se observa en imágenes de mayor tamaño. Esto indica que el riesgo de sobreajuste es considerablemente menor, permitiendo a la red neuronal generalizar mejor.

Por lo tanto, observamos que, si bien las imágenes de mayor tamaño permiten a la red capturar más información y alcanzar un rendimiento perfecto en entrenamiento, su capacidad de generalización se ve empeorada a medida que el tamaño crece, debido al sobreajuste evidente en la validación. En este caso, los tamaños de imagen más moderados, como 64x64, nos ofrecen el mejor balance entre ajuste al conjunto de entrenamiento y capacidad de generalización, mientras que imágenes de menor tamaño.

1.3 Estudie la variación en rendimiento para diferentes funciones de activación durante 25 épocas. Seleccione 3-4 opciones y, salvo la última capa con activación softmax, cambie TODAS las funciones de activación de la red en las distintas capas a las opciones seleccionadas. Como inicialización de parámetros, aplique el valor por defecto en cada capa. Utilice un valor de batch_size e img_size acorde sus conclusiones en las preguntas 4.1 y 4.2. Razone porque obtiene los resultados que observa.

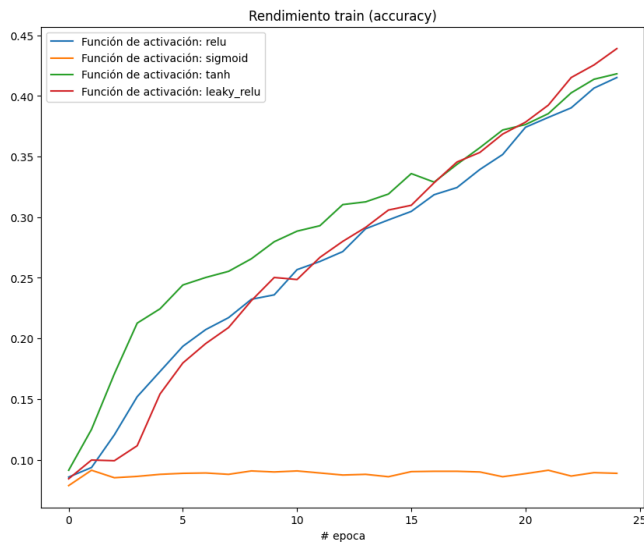


Fig. 9. Precisión en train para diferentes funciones de activación

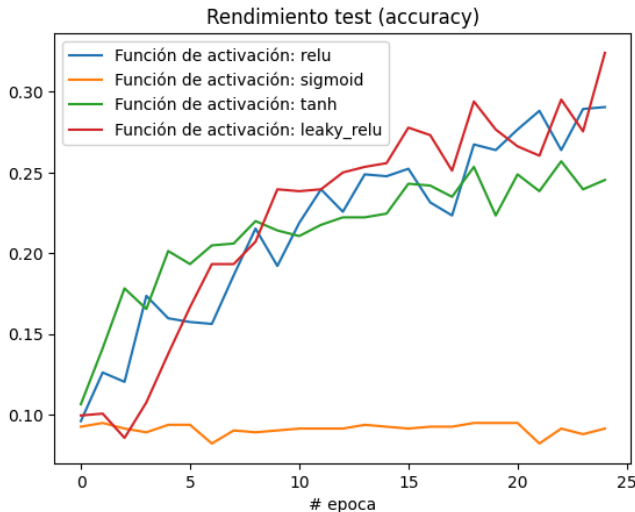


Fig. 10. Precisión en test para diferentes funciones de activación

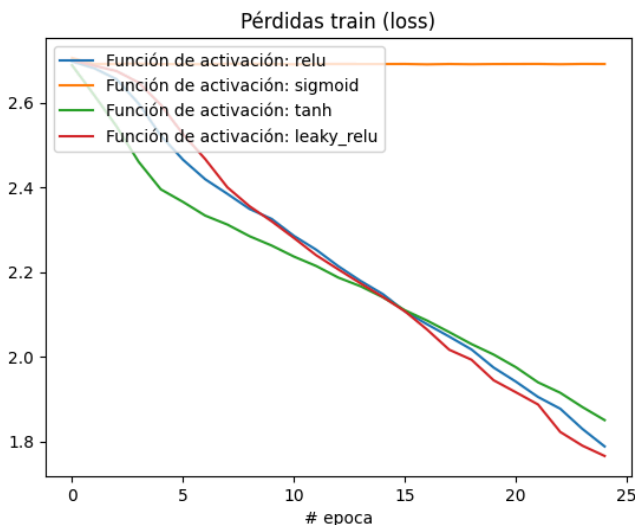


Fig. 11. Pérdidas en train para diferentes funciones de activación

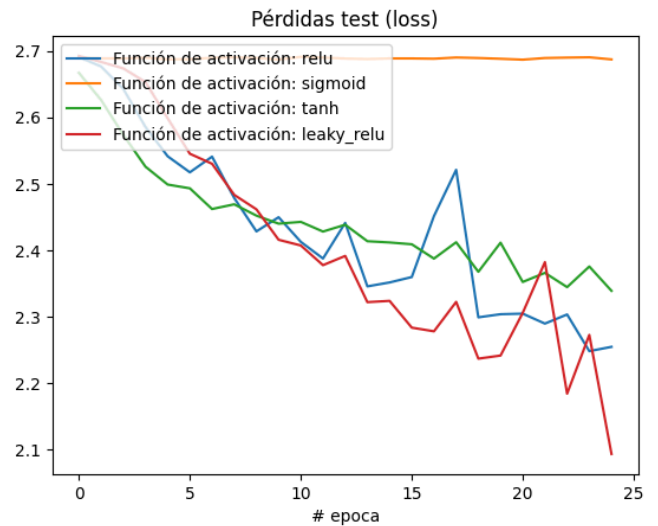


Fig. 12. Pérdidas en test para diferentes funciones de activación

Empleando el tamaño de batch 32 y el tamaño de imagen 64x64, estudiamos el impacto de diferentes funciones de activación en el rendimiento de la red neuronal durante 25 épocas, para ello hemos considerado las funciones ReLU, Leaky ReLU, tanh y sigmoid, manteniendo la activación softmax en la última capa.

Los resultados obtenidos muestran diferencias tanto en el rendimiento como en la pérdida entre las distintas funciones de activación. En el conjunto de entrenamiento, el rendimiento de la función sigmoide se mantiene como una línea horizontal sin mostrar ninguna mejora a lo largo de las épocas, este comportamiento refleja que el aprendizaje no está ocurriendo de manera efectiva debido seguramente a la saturación que ocurre con las funciones sigmoides.

Por otro lado, las funciones tanh, ReLU y Leaky ReLU presentan un comportamiento que no permanece constante. En las primeras épocas, tanh muestra un rendimiento ligeramente superior al de ReLU y Leaky ReLU, evidenciando que su capacidad para producir salidas centradas en cero proporciona cierta ventaja inicial. Sin embargo, a partir de la época 15, tanh pierde esta ventaja y su rendimiento se iguala al de ReLU, terminando en niveles similares al cabo de las 25 épocas. En contraste, Leaky ReLU no solo comienza de manera similar a ReLU, sino que termina superándola en el rendimiento hacia el final del entrenamiento, demostrando ser la mejor opción entre las cuatro funciones evaluadas.

En el conjunto de validación, las tendencias son similares, aunque con ligeras variaciones. Sigmoid nuevamente no muestra mejora alguna, Tanh empieza mejor que ReLU en las primeras épocas, pero al final del entrenamiento, ReLU consigue superarla en rendimiento, y como en el entrenamiento, Leaky ReLU mantiene un desempeño superior, logrando la mejor capacidad de generalización entre las funciones evaluadas.

En cuanto a las pérdidas, los resultados son consistentes con el rendimiento observado en ambos conjuntos,

entrenamiento y validación. Para sigmoide, la pérdida permanece completamente horizontal en ambos casos, indicando que la red no está aprendiendo ni ajustándose a los datos. Las pérdidas asociadas a tanh, ReLU y Leaky ReLU son inicialmente similares, con tanh mostrando una ligera ventaja al comienzo. Sin embargo, hacia la época 15, las pérdidas de tanh se igualan con las de ReLU y, al final del entrenamiento, tanh resulta ser la peor de las tres, aunque sin una diferencia significativa. Por su parte, Leaky ReLU mantiene una mejora constante y termina siendo la mejor, con la pérdida más baja tanto en entrenamiento como en validación.

Estos resultados pueden explicarse por las características propias de cada función de activación. La función Sigmoide, aunque adecuada para problemas simples, sufre problemas de saturación y desvanecimiento del gradiente, lo que impide un aprendizaje efectivo. Tanh proporciona una mejor capacidad de aprendizaje inicial gracias al centrado en cero, pero también está limitada por problemas de saturación en valores extremos. ReLU y Leaky ReLU, al no saturarse en valores positivos y al mantener gradientes para valores negativos (en el caso de Leaky ReLU), permiten un uso de los gradientes más efectivo. La ventaja de Leaky ReLU sobre ReLU radica en su capacidad para evitar el problema de neuronas muertas, asegurando un aprendizaje continuo.

En conclusión, Leaky ReLU es la mejor función de activación para este caso, tanto en términos de rendimiento como de pérdida en los conjuntos de entrenamiento y validación. Por otro lado también como hemos observado hay que destacar que Tanh puede ser útil para etapas iniciales del aprendizaje, pero hay que tener en cuenta la caída de rendimiento con el tiempo, mientras que sigmoid demuestra ser inadecuada para esta configuración debido a su incapacidad de aprender de manera efectiva. [1]

1.4 Estudie la variación en rendimiento para diferentes opciones de Data Augmentation (consulta ImageDataGenerator) durante 25 épocas. Seleccione 3-4 opciones, argumente su elección y compare el rendimiento obtenido con la red original del tutorial (que no aplica Data Augmentation). Utilice un valor de batch_size e img_size acorde sus conclusiones en las preguntas 4.1 y 4.2. Razone porque obtiene los resultados que observa.

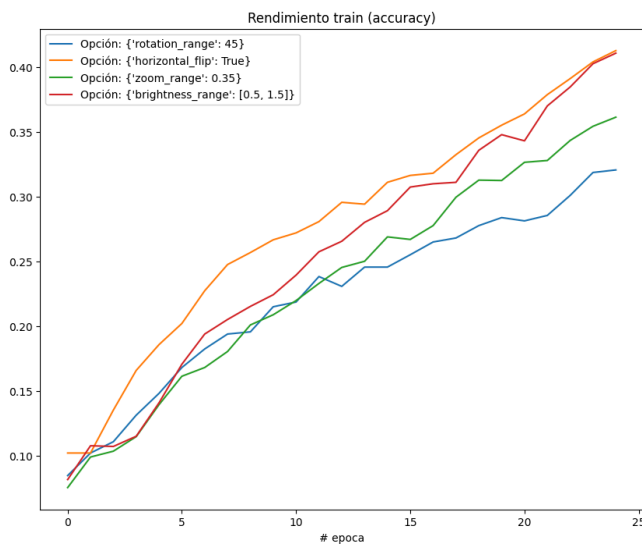


Fig. 12. Precisión en train para diferentes configuraciones de DataAugmentation

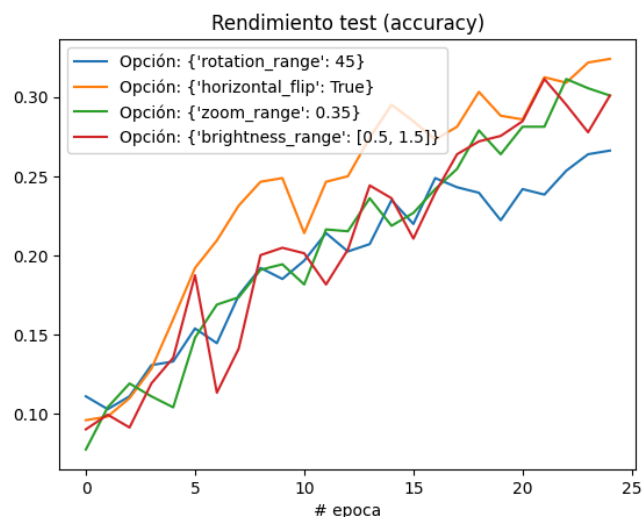


Fig. 13. Precisión en test para diferentes configuraciones de DataAugmentation

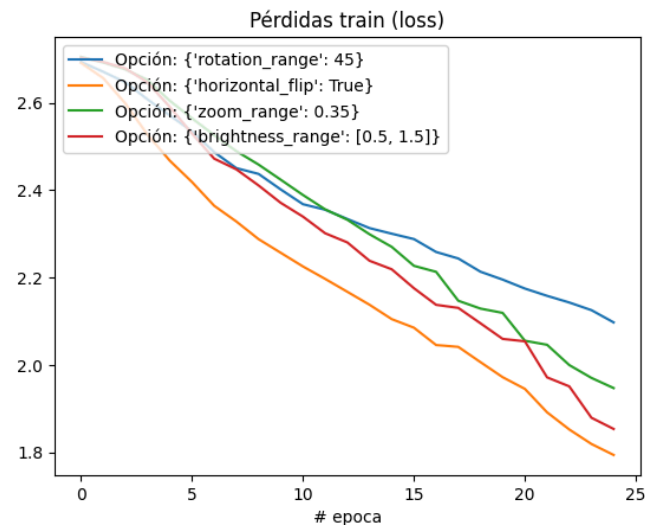


Fig. 14. Pérdidas en train para diferentes configuraciones de DataAugmentation

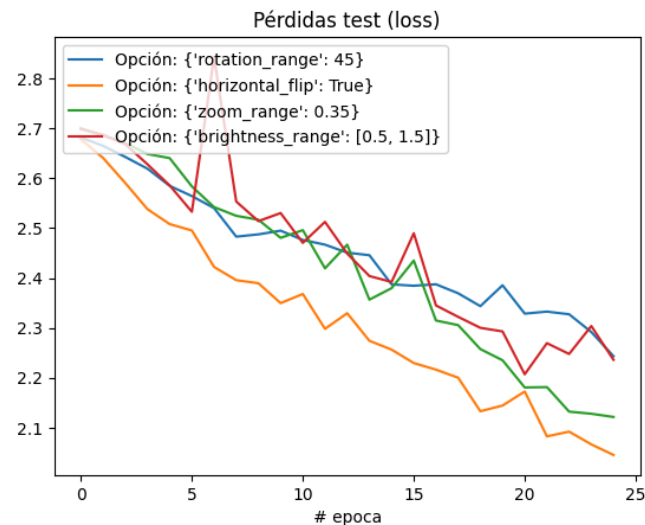


Fig. 15. Pérdidas en test para diferentes configuraciones de DataAugmentation

El uso de Data Augmentation, empleando una rotación de 45 grados, un flip horizontal, un zoom de 0.35 y un rango de brillo de 0.5 a 1.5 [2], introduce diferencias notables en el comportamiento de la red neuronal en comparación con la configuración original del tutorial. En el conjunto de entrenamiento, la precisión no alcanza valores cercanos a 1 para ninguna de las configuraciones de Data Augmentation, situándose entre 0.3 y 0.4 al final de las 25 épocas, esto contrasta con la red original, que logra una precisión perfecta en entrenamiento. Este comportamiento refleja el impacto de la mayor complejidad y diversidad introducidas en los datos, que dificultan el ajuste completo al conjunto de entrenamiento, reduciendo así el riesgo de sobreajuste.

En el conjunto de validación, el rendimiento en términos de precisión es prácticamente igual al de la red del tutorial, mostrando que las técnicas de Data Augmentation no afectan negativamente la capacidad de generalización en este aspecto. Sin embargo, las pérdidas presentan diferencias significativas, ya que en la red

original, las pérdidas en validación muestran un claro sobreajuste, con un incremento hacia el final de las épocas, por el contrario, en las configuraciones con Data Augmentation, las pérdidas en validación continúan decreciendo de manera constante a lo largo de las 25 épocas. Este resultado indica que la inclusión de variaciones en los datos de entrenamiento no solo evita el sobreajuste, sino que también mejora la capacidad de la red para generalizar.

Estas diferencias pueden explicarse por el efecto del Data Augmentation al ampliar la diversidad del conjunto de entrenamiento, obligando a la red a aprender características más generales y diversas en lugar de memorizar patrones específicos de los datos. Cada técnica contribuye de manera distinta: la rotación mejora la invariancia a cambios de orientación, el flip horizontal introduce simetría en las imágenes, el zoom amplía la capacidad de reconocer objetos a diferentes escalas, y las variaciones de brillo permiten manejar mejor condiciones de iluminación variadas. Esta mayor diversidad de datos aumenta la dificultad del aprendizaje en entrenamiento, como se observa en la precisión más baja en este conjunto, pero al mismo tiempo mejora la capacidad de generalización, como lo evidencia la disminución sostenida de la pérdida en validación.

4 CARGA DE TRABAJO

Tarea	Horas dedicadas (Carlos García Santa)	Horas dedicadas (Miguel Ángel González Gallego)
Tarea 1	1	0.5
Memoria	4.5	3

Tabla 1. Carga de trabajo en horas

REFERENCIAS

[1] Juan Carlos San Miguel Avedillo, Tema 4: Procesamiento de imágenes con Aprendizaje Profundo (Deep Learning) - Diapositivas 40 a 44.
[2] Juan Carlos San Miguel Avedillo, Tema 4: Procesamiento de imágenes basado en Aprendizaje Profundo (Deep Learning) Entrenamiento de redes CNN - Diapositivas 49 a 51.